

Exercise sheet 2 for Numerical Optimization

Dr. Michael Lehn

Tobias Born

Deadline: October 29, 2025

Exercise 1 (Levenberg-Marquardt method, 5+5+4+4+6+2 points)

We are still dealing with nonlinear least squares problems. For their definition and the notation, see exercise sheet 1. We will now introduce a second approach for nonlinear least squares problems, the Levenberg-Marquardt method. It corresponds to the Gauß-Newton method with the slight difference, that $g''(x_k)$ is approximated in a different way. Analytically, it is

$$g''(x_k) = F'(x_k)^T F'(x_k) + \sum_{l=1}^m F_l(x_k) F_l''(x_k) .$$

The Gauß-Newton approximation is

$$g''_{(\text{GN})}(x_k) = F'(x_k)^T F'(x_k) . \quad (1)$$

For the Levenberg-Marquardt method, we define

$$g''_{(\text{LM})}(x_k) = F'(x_k)^T F'(x_k) + \mu^2 I_n$$

with $\mu \geq 0$. Therefore, the Newton correction turns out to be

$$s_k = - (F'(x_k)^T F'(x_k) + \mu^2 I_n)^{-1} F'(x_k)^T F(x_k) . \quad (2)$$

- a) Analogously to sheet 1 exercise 3b, to which linear least squares problem is (2) equivalent?
- b) Show that this linear least squares problem corresponds to the linear least squares problem (8) from sheet 1 plus a penalty term on s_k .

For $\mu = 0$ in every iteration step, the linear least squares problem (8) from sheet 1 is solved and the Gauß-Newton method executed (obviously). For $\mu > 0$, the same idea as for Gauß-Newton is adopted, but at the same time, the Newton correction is prevented from becoming too large. The basic idea is that one supposes that the error caused by approximating $g''(x_k)$ with $g''_{(\text{GN})}(x_k)$, see (1), has no significant impact in a neighbourhood of x_k . Or, conversely, that the approximation error has a significant impact if one moves too far away from x_k . The penalty term is intended to ensure that s_k is small enough so that $x_{k+1} = x_k + s_k$ is so close to x_k that the approximation error is acceptably small.

- c) Show the damping property of μ on the update s_k by proving that

$$\|s_k\|_2 \leq \frac{\|F(x_k)\|_2}{\mu} .$$

So the bigger μ is, the bigger is the penalty term and the smaller is the neighbourhood in which x_{k+1} can be.

- d) For real matrices A it holds true that $\text{rank}(A) = \text{rank}(A^T A)$. So for the Gauß-Newton method, it is necessary that $F'(x_k) \in \mathbb{R}^{m \times n}$ with $m \geq n$ has rank n in order to guarantee that the $(n \times n)$ -dimensional system matrix $g''_{(\text{GN})}(x_k)$, (1), for the update is invertible. This has to be assumed to hold true for every iterate x_k , $k = 0, 1, \dots$.
How is it with the Levenberg-Marquardt method, i.e. for $\mu > 0$?

Now the question arises as to how the damping parameter μ should be selected. If it is too small, the iteration can leave the trusted neighbourhood and the approximation potentially becomes bad. If it is too big, the update is very small and the iteration progresses slowly. So it is about finding a good trade-off. At the same time, there is no reason to assume that a choice for μ that was a good choice in one iteration step will also be a good choice in the following iteration step. For this reason, often a damping strategy is used, the principle of which is very similar to the damping used in the damped Newton's method (recall Numerical Analysis). To do this, for a potential update s_k , it is examined how large of an error the linearization produces. The change in the objective function induced by the update s_k

$$\|F(x_k)\|_2^2 - \|F(x_k + s_k)\|_2^2$$

is compared with the change in the linearization of the objective function

$$\|F(x_k)\|_2^2 - \|F(x_k) + F'(x_k)s_k\|_2^2.$$

To this end, define the indicator

$$\epsilon_\mu := \frac{\|F(x_k)\|_2^2 - \|F(x_k + s_k)\|_2^2}{\|F(x_k)\|_2^2 - \|F(x_k) + F'(x_k)s_k\|_2^2}. \quad (3)$$

If s_k is a reasonable choice for the update, it generates an improvement in both the actual and the linearized objective function, so we assume that $\epsilon_\mu > 0$. If the linearization was exact, it would be $\epsilon_\mu = 1$. The basic idea is as follows:

- If ϵ_μ is very small, the linearization error is too large and the update is rejected. Increase μ in order to reduce the trusted neighbourhood to enforce a better approximation error.
- Otherwise, accept the update.
- If ϵ_μ is very large, the linearization error is so small that the trusted neighbourhood can be enlarged to allow larger steps, therefore decrease μ .

With $0 < \beta_0 < \beta_1 < 1$ this means:

- $\epsilon_\mu \leq \beta_0$: reject s_k , set $\mu \leftarrow 2\mu$
- $\beta_0 < \epsilon_\mu < \beta_1$: accept s_k
- $\epsilon_\mu \geq \beta_1$: accept s_k , set $\mu \leftarrow \mu/2$

The parameters β_0 and β_1 are purely heuristic and must be selected a priori. The resulting algorithm can be seen in Algorithm 1, keep in mind that it is heuristic.

Algorithm 1: Levenberg-Marquardt method

```
1 Input: initial value  $x_0 \in \mathbb{R}^n$ ,  $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ ,  $F' : \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}$ , initial damping
   parameter  $\mu_0, \beta_0, \beta_1$ 
2 Output: approximation  $x_k$  of a solution to the respective nonlinear least squares
   problem
3 for  $k \leftarrow 0, 1, \dots$  do
4   compute  $F_{(k)} \leftarrow F(x_k)$  and  $F'_{(k)} \leftarrow F'(x_k)$ 
5   solve system of linear equations  $s_k \leftarrow - \left( (F'_{(k)})^T F'_{(k)} + \mu^2 I_n \right)^{-1} (F'_{(k)})^T F_{(k)}$ 
6   compute  $\epsilon_\mu$  according to (3)
7   while  $\epsilon_\mu \leq \beta_0$  do
8      $\mu \leftarrow 2\mu$ 
9     solve system of linear equations  $s_k \leftarrow - \left( (F'_{(k)})^T F'_{(k)} + \mu^2 I_n \right)^{-1} (F'_{(k)})^T F_{(k)}$ 
10    compute  $\epsilon_\mu$  according to (3)
11  if  $\epsilon_\mu \geq \beta_1$  then
12     $\mu \leftarrow \mu/2$ 
13   $x_{k+1} \leftarrow x_k + s_k$ 
```

- e) Create a MATLAB programme `levenberg_marquardt.m` that gets inputs `x0`, function handles `F` and `dF`, an initial damping parameter `mu0`, `beta0`, `beta1`, a tolerance `tol` as well as a maximum number of iteration steps `maxit` and implements the Levenberg-Marquardt method. The programme is supposed to terminate when either $\|s_k\|_2 < \text{tol}$ or when `maxit` iteration steps have been carried out.

Note: You can use the MATLAB operator `\` in order to solve the system of linear equations. But keep in mind that a big part of the lecture Numerical Linear Algebra dealt with this kind of problems.

Note: The programme must return a matrix of dimension $(n \times \text{number iterations} + 1)$ containing all iterates. Also it must return a $(\text{number iterations} + 1)$ -dimensional vector containing the applied damping parameters of all iteration steps.

- f) The MATLAB programmes `test_rosenbrock.m` and `error_rosenbrock.m` correspond to the programmes of the same name in sheet 1, except that this time the Rosenbrock function is optimized with both Gauß-Newton and Levenberg-Marquardt and that also the damping parameters of Levenberg-Marquardt are plotted over the iteration steps. The optimizations are performed for two different initial values. The parameters for the damping strategy of Levenberg-Marquardt are chosen as

$$\mu_0 = 0.3, \quad \beta_0 = 0.3, \quad \beta_1 = 0.9.$$

Execute the programmes and reflect on the plots.

Exercise 2 (Newton-Raphson method for easy examples, 6+2 points)

By leaving the update s_k as it comes from Newton's method, i.e.

$$s_k = - \left(F'(x_k)^T F'(x_k) + \sum_{l=1}^m F_l(x_k) F_l''(x_k) \right)^{-1} F'(x_k)^T F(x_k) ,$$

one gets the Newton-Raphson method. In complicated situations, we do not want to use this method due to the m Hessian matrices. But we will take a look at it for very easy examples with one-dimensional functions $F : \mathbb{R} \rightarrow \mathbb{R}$.

- a) Create a MATLAB programme `newton_raphson.m` that gets a starting point `x0`, a function handle `F`, a function handle `dF` for the derivative, a function handle `ddF` for the second derivative, a tolerance `tol` and a maximum number of iteration steps `maxit` and implements the Newton-Raphson method. The termination criterion is the same as for the Levenberg-Marquardt method. The programme is supposed to return a vector containing all iterates.
- b) The MATLAB programme `test_newton_raphson.m` applies Gauß-Newton, Levenberg-Marquardt and Newton-Raphson to two simple examples,

$$f_1(x) = \|x^2 + 1\|_2^2 \quad \text{and} \quad f_2(x) = \|x^2\|_2^2 .$$

Run the programme and reflect on the results. In particular, focus on the behaviour of Newton-Raphson.

Exercise 3 (Heuristic approaches: Particle Swarm Optimization, 6+1+2+2+2+2+1)

Numerical optimization has a fundamental problem. Unless the function under consideration has special properties (e.g. convexity, ...), it is impossible to determine its global minimum with complete certainty. Take an arbitrary function and try to determine its global minimum. No matter what method you use and what the result is, how can you be sure that there is no point somewhere on the domain where the function adopts an even smaller value? For this reason, we will only deal with local optimization within the lecture, i.e., the search for local minima.

If one still wants to try to find a point where the function is globally as small as possible, you have no choice but to use heuristic approaches. These are based on ideas that sometimes work better and sometimes work worse. Except for special cases, it is not possible to figure out how wrong their results are. So, evaluating their results is less a question of mathematics and more a question of experience. Nevertheless, these approaches are common and widespread, as there is no other option. In practice, these approaches are normally used as a way to find promising starting values for local optimizations.

To enrich the lecture and so that you have seen such approaches, we will occasionally look at such methods on the exercise sheets, detached from the context of the lecture.

On this sheet, we will take a look at an algorithm called *Particle Swarm Optimization*. The name comes from the fact that it is intended to mimic the behaviour of animal swarms in a simple way. The actors are a population of particles that move around the search area. In each iteration step, each particle moves depending on its previous movement direction, the best function value it has found so far and the best function value that the entire population has found so far.

The procedure is as follows: First, random starting positions $p_0^{(1)}, \dots, p_0^{(n)}$ and random starting velocities $v_0^{(1)}, \dots, v_0^{(n)}$ are drawn for n particles. For each particle, a record is kept of its position with the best function value across all iterations. At each iteration step, it must therefore be checked whether the new position has a function value that is better than the particle's former best position. The minimum of the i -th particle up to the k -th iteration step is denoted by $\bar{p}_k^{(i)}$. Similarly, the current minimum across all particles after k iteration steps, i.e., the current global minimum, is denoted by $\bar{g}_k$ and must be updated in each iteration step. In the k -th iteration step, the velocity $v_k^{(i)}$ is determined for the i -th particle. It is composed of the previous velocity multiplied by an inertia ω , the distance of the particle to its minimum so far multiplied with a specified weight c_p and a random weight and the distance of the particle to the global minimum so far multiplied with a specified weight c_g and a random weight. The new position $p_k^{(i)}$ is then determined by adding the velocity $v_k^{(i)}$ to the old position $p_{k-1}^{(i)}$.

Algorithm 2: Particle Swarm Optimization

- 1 **Input:** initial positions $p_0^{(i)}$ and velocities $v_0^{(i)}$, $i = 1, \dots, n$, inertia ω , weights c_p and c_g
 - 2 **Output:** approximation of the global minimum $\bar{g}_k$
 - 3 **for** $k \leftarrow 1, 2, \dots$ **do**
 - 4 update $\bar{p}_{k-1}^{(i)}$, $\forall i = 1, \dots, n$ // particle's best position over all iteration steps
 - 5 determine $\bar{g}_{k-1}$ // global best position over all iteration steps
 - 6 $v_k^{(i)} \leftarrow \omega v_{k-1}^{(i)} + \text{rand} \cdot c_p (\bar{p}_{k-1}^{(i)} - p_{k-1}^{(i)}) + \text{rand} \cdot c_g (\bar{g}_{k-1} - p_{k-1}^{(i)})$, $\forall i = 1, \dots, n$
 - 7 where rand is a uniformly distributed random number in $(0, 1)$
 - 8 $p_k^{(i)} = p_{k-1}^{(i)} + v_k^{(i)}$, $\forall i = 1, \dots, n$
-

The algorithm should terminate when either a maximum number of iteration steps has been reached (**max_it**) or the global optimum $\bar{g}_k$ has not changed for a certain number of iteration steps (**max_noupd**).

- a) We want to implement the Particle Swarm Optimization. Therefore, fill the gaps in the MATLAB programme `particle_swarm_optimization.m`.

We want to try out `particle_swarm_optimization.m` on the functions

$$f_1(x, y) = x^2 + y^2 + 10(2 - \cos(2\pi x) - \cos(2\pi y)) \quad \text{and}$$

$$f_2(x, y) = 1 + \frac{x^2 + y^2}{4000} - \cos(x) \cos\left(\frac{y}{\sqrt{2}}\right).$$

- b) Execute the MATLAB programme `visualize_functions.m` and understand what f_1 and f_2 look like and which of their properties could be relevant for optimization.

From now on, the heuristic parameters are always selected as $\omega = 0.7$, $c_p = 1.5$, $c_g = 1.5$, `max_it` = 200 and `max_noupd` = 20. Keep in mind that these parameters could also be selected completely differently and that the algorithm may then behave completely differently.

- c) The MATLAB programme `test_pso_f1.m` draws 30 random particles, uniformly distributed on $(-10, 10) \times (-10, 10)$, in combination with random initial velocities uniformly distributed on $(-4, 4) \times (-4, 4)$. Then it tries to optimize f_1 using `particle_swarm_optimization.m` and plots the results.
Run the programme several times and reflect on the outcome.
- d) The MATLAB programme `test_pso_f2.m` does exactly the same with f_2 . Run it several times and reflect on the outcome.

As we have no information about the error of the method, it is not useful to perform it only once. Instead, the method will be performed several times for different random initial values and the results will be compared.

- e) The MATLAB programme `test_pso_f2_repeat.m` does the same as `test_pso_f2.m`, with the difference that it performs ten independent optimization runs, each with different random starting values.
Run the programme and reflect on the outcome.
- f) Finally, let us also take a look at the Rosenbrock function. The MATLAB programme `test_pso_rosenbrock.m` applies `particle_swarm_optimization.m` with 30 uniformly distributed initial positions in $(-5, 5) \times (-5, 5)$ and uniformly distributed initial velocities in $(-2, 2) \times (-2, 2)$ to the Rosenbrock function.
Run the programme and reflect on the outcome.
- g) Finally, I encourage you to play around with the heuristic parameters of the Particle Swarm Optimization to get a feeling of how different it behaves for differing choices of parameters.